AI Content Copilot

Frontend Maintenance & Developer Guide

How the frontend was built, how to maintain it, and how to add or change things.

Table of Contents

1. What This Guide Covers
2. Technology Stack
3. Project Structure (Complete File Map)
4. How the Pieces Fit Together
5. The Styling System
6. How Pages Work
7. Adding a New Page
8. Modifying an Existing Page
9. How the Website Talks to the Backend (API Layer)
10. Adding a New Backend Connection
11. The Navigation System
12. Adding a New Navigation Link
13. Reusable Components
14. Environment Settings
15. Running the Website Locally
16. How Deployments Work
17. Maintaining the App with AI
18. Quick Reference Cheat Sheet

1. What This Guide Covers

This doc explains how the AI Content Copilot frontend was built, how all the pieces fit together, and most importantly, how to add new features, change existing ones, or fix issues.

It's written so that even if you're not a developer, you can understand what a developer would need to do. You can also hand this to an AI coding assistant (like GPT-5 or Claude) and it'll have enough context to make changes.

What you'll learn:

- What each file in the project does
- How to add a brand-new page to the website
- How to change text, labels, or behaviour on existing pages
- How to connect a new backend workflow (n8n) to the website
- How to add a new link in the sidebar
- How to change the look and feel (colours, fonts, spacing)
- How to deploy changes
- How to use AI coding assistants to maintain the app

2. Technology Stack

We're using standard, widely-supported technologies here. Any competent web developer can work on this, and AI coding tools are highly trained on this exact stack.

Technology	What It Is	Why We Use It
React 19	Framework for building interactive websites	The most popular web framework out there. Huge community, easy to hire for
Vite 7	Build tool that compiles the code	Very fast dev experience, industry standard
React Router 7	Handles page navigation (URL to page)	Standard way to manage multiple pages in React
Vanilla CSS	Custom styling (colours, layout, fonts)	Maximum flexibility, no dependency on CSS frameworks
Vercel	Website hosting and deployment	Free tier available, auto-deploys from GitHub

Why this setup: *We went with vanilla CSS instead of something like Tailwind because it's simpler for this project's size and doesn't add another dependency. If the app grows significantly, a component library might make sense down the line.*

3. Project Structure (Complete File Map)

Here's every file and folder in the project, with a plain description of what each one does.

Root Files (Configuration)

File	What It Does	When You'd Touch It
.env	Stores settings (n8n URL, mock mode)	When changing which backend to connect to, or toggling mock data
.env.example	Template showing what settings are available	Never, it's just a reference for new devs
vercel.json	Tells Vercel how to route requests (proxies /api/n8n to n8n Cloud)	When changing the n8n Cloud URL
vite.config.js	Configures the build tool (also sets up the local proxy)	When changing the n8n Cloud URL for local dev
package.json	Lists all dependencies and scripts (npm install, npm run dev)	When adding a new library
index.html	The HTML shell that React fills in	Rarely, only to change the page title or meta tags

Source Code (src/ folder)

Core Files

File	What It Does	When You'd Touch It
src/main.jsx	The entry point, loads React and renders the app	Rarely, only if changing app-wide wrappers
src/App.jsx	Defines ALL pages and their URL paths (routing)	When adding a new page
src/index.css	ALL visual styling: colours, fonts, buttons, cards, layout	When changing how anything looks

Pages (src/pages/)

Each file here is one full page of the website:

File	URL Path	What The Page Does
Home.jsx	/	Landing page with hero section and feature cards
Dashboard.jsx	/dashboard	Overview with stats and recent activity feed
AnalyzeKeyword.jsx	/analyze	Form to analyze keywords and get PAVE scores from AI
ContentBacklog.jsx	/backlog	Table showing all saved keywords with status filters
CreateBrief.jsx	/brief	Step-by-step wizard to create AI authority briefs
ProduceContent.jsx	/produce	Generate full HTML articles with progress tracking
Published.jsx	/published	Gallery grid of all published articles

Components (src/components/)

Reusable building blocks used across multiple pages:

File	What It Is	Used By
layout/MainLayout.jsx	The page shell - combines Navbar + Sidebar + page content	All pages except Home
layout/Navbar.jsx	Top navigation bar with logo and user info	MainLayout (appears on every page)
layout/Sidebar.jsx	Left sidebar with navigation links	MainLayout (appears on every page)
common/LoadingSpinner.jsx	A spinning animation shown while data loads	AnalyzeKeyword, Backlog, Brief, Produce, Published
common/PAVEScoreCard.jsx	The card showing P/A/V/E scores with editable inputs	AnalyzeKeyword page
common/StatusBadge.jsx	Coloured badge showing item status (e.g. 'Briefed')	ContentBacklog page

Services (src/services/)

File	What It Does	When You'd Touch It
api.js	ALL backend communication, every n8n webhook call lives here	When adding a new backend feature or changing how data flows

Hooks (src/hooks/)

File	What It Does	When You'd Touch It
useApi.js	Helper that handles loading states and errors for API calls	Rarely, it's a utility used by pages

4. How the Pieces Fit Together

Here's how the website works from top to bottom:

User visits the website > The browser loads index.html, which loads main.jsx

main.jsx starts React > It wraps everything in a Router (for page navigation)

App.jsx defines the pages > It maps URL paths to page components (e.g., /analyze goes to AnalyzeKeyword.jsx)

MainLayout wraps internal pages > It adds the Navbar (top bar) and Sidebar (left menu) around page content

Pages use api.js to get data > When a page needs data, it calls a function in api.js

api.js talks to n8n > It sends requests to the n8n backend through the Vercel proxy

Data comes back, page updates > React automatically re-renders with the new data

The key thing to know: each page is independent. You can change AnalyzeKeyword.jsx without affecting ContentBacklog.jsx. The only shared pieces are the layout (Navbar, Sidebar) and the API service (api.js).

We didn't use a state management library like Redux because the app's simple enough that it doesn't need one. Each page manages its own state. If the app grows to the point where pages need to share a lot of data, that might change.

5. The Styling System

All visual styling lives in one file: src/index.css. It's about 600+ lines, controlling every colour, font, spacing, animation, and layout in the app.

How Styles Are Organized

Section	What It Controls	Examples
CSS Variables (top)	Colour palette, font sizes, spacing values	--color-primary, --font-size-sm, --space-4
Base Styles	Default text, body background, links	Body font, background colour
Layout	Page structure, sidebar width, navbar height	.main-layout, .sidebar, .navbar
Cards & Containers	Content boxes used across pages	.card, .table-container
Buttons	All button styles and variants	.btn-primary, .btn-secondary, .btn-ghost
Forms	Input fields, dropdowns, textareas	.input, .select, .label
Tables	Data table styling	table, th, td
Badges	Status indicators (coloured pills)	.badge-success, .badge-warning
Animations	Fade-ins, spinners, transitions	.animate-fade-in, .spinner
Page-Specific	Styles for specific pages	.landing-hero, .gallery-grid, .pave-grid

How to Change the Colour Scheme

All colours are defined as CSS Variables at the top of index.css. To change the colour scheme, you just update these variables and everything updates automatically.

Colour Variables in index.css:

```
/* Find these at the top of src/index.css */
:root {
    --color-primary: #6366F1;       /* Main purple - buttons, links, accents */
    --color-primary-hover: #5558E6; /* Button hover state */
    --color-success: #10B981;       /* Green - success messages, badges */
    --color-warning: #F59E0B;       /* Amber - warnings */
    --color-error: #EF4444;         /* Red - errors */
    --bg-primary: #0A0A1A;          /* Main background colour */
    --text-primary: #FFFFFF;        /* Main text colour */
}
```

Example: *To change the brand colour from purple to blue, you'd just swap --color-primary from #6366F1 to something like #3B82F6. Every button, link, and accent in the app updates.*

6. How Pages Work

Every page follows the same pattern. Once you understand it, you can read (and modify) any page.

The pattern:

1. Import tools: The page imports React hooks, API functions, and any components it needs
2. Define state: Variables that track what's happening (is data loading? what did the user type?)
3. Define actions: Functions that run when the user clicks a button

4. Return layout: The HTML-like structure that defines what appears on screen

Page Pattern Template:

```
// Simplified page structure (every page looks like this)

import { useState } from 'react'           // 1. Import tools
import { api } from '../services/api'

export default function MyPage() {
    const [data, setData] = useState(null)  // 2. Define state

    const handleClick = async () => {       // 3. Define actions
        const result = await api.someFunction()
        setData(result)
    }

    return (                                // 4. Layout
        <div>
            <h1>My Page Title</h1>
            <button onClick={handleClick}>Do Something</button>
            {data && <p>{data.message}</p>}
        </div>
    )
}
```

7. Adding a New Page

Adding a page requires changes in exactly 3 files. That's it.

Step 1: Create the Page File

Create a new file in src/pages/. Say you want a "Settings" page:

Create: src/pages/Settings.jsx

```
// File: src/pages/Settings.jsx

export default function Settings() {
    return (
        <div className="animate-fade-in">
            <div className="page-header">
                <h1 className="page-title">Settings</h1>
                <p className="page-subtitle">Manage your account settings</p>
            </div>

            <div className="card">
                <h3>Your Settings Go Here</h3>
                <p>Add forms, toggles, or whatever you need.</p>
            </div>
        </div>
    )
}
```

Step 2: Register the Route

Open src/App.jsx and add your new page:

Edit: src/App.jsx

```
// In src/App.jsx, add these two things:

// 1. Add this import at the top:
import Settings from './pages/Settings'

// 2. Add this route inside the <Route element={<MainLayout />}> block:
<Route path="/settings" element={<Settings />} />
```

Step 3: Add to the Sidebar

Open src/components/layout/Sidebar.jsx and add a new link:

Edit: src/components/layout/Sidebar.jsx

```
// In src/components/layout/Sidebar.jsx
// Add this entry to the 'navigation' array:

const navigation = [
    { name: 'Dashboard', href: '/dashboard', icon: 'chart' },
    { name: 'Analyze Keyword', href: '/analyze', icon: 'search' },
    { name: 'Content Backlog', href: '/backlog', icon: 'list' },
    { name: 'Create Brief', href: '/brief', icon: 'edit' },
    { name: 'Produce Content', href: '/produce', icon: 'zap' },
    { name: 'Published', href: '/published', icon: 'rocket' },
    { name: 'Settings', href: '/settings', icon: 'gear' }   // <-- NEW
]
```

Done! *That's it. Three files, three simple changes. Push to GitHub and Vercel auto-deploys the update.*

8. Modifying an Existing Page

Common modifications and where to make them:

What You Want to Change	File to Edit	What to Look For
Page title or description	src/pages/[PageName].jsx	Look for <h1 className="page-title"> and <p className="page-subtitle">
Button text	src/pages/[PageName].jsx	Look for <button> elements
Form fields (add/remove/rename)	src/pages/[PageName].jsx	Look for <input>, <select>, or <textarea> elements
The status filter options on Backlog	src/pages/ContentBacklog.jsx	Edit the STATUS_FILTERS array at the top
Dashboard statistics	src/pages/Dashboard.jsx	Edit the sections
Production progress steps	src/pages/ProduceContent.jsx	Edit the PRODUCTION_STEPS array at the top
PAVE scoring display	src/components/common/PAVEScoreCard.jsx	Edit the score labels, ranges, or layout
Colours, fonts, spacing	src/index.css	Edit the CSS variables at the top
Sidebar navigation links	src/components/layout/Sidebar.jsx	Edit the navigation array
Navbar logo or title	src/components/layout/Navbar.jsx	Edit the text in the navbar-brand div

9. How the Website Talks to the Backend (API Layer)

All communication between the website and n8n goes through one file: src/services/api.js. It has a class called ApiService with methods for every backend operation.

How It Works

- 1. A page calls a function like api.analyzeKeyword({...})
- 2. api.js checks if mock mode is on (VITE_USE_MOCK_DATA)
- 3. If mock mode: returns fake data from built-in arrays (for testing)
- 4. If real mode: sends an HTTP request to /api/n8n/webhook/[WorkflowName]
- 5. Vercel's proxy forwards /api/n8n/... to https://vitcornu.app.n8n.cloud/...
- 6. n8n processes the request and sends data back
- 7. api.js transforms the response into the format the page expects
- 8. The page gets the data and displays it

API Method Reference

Method Name	n8n Webhook	HTTP Method	What It Does
getBrands()	/webhook/GetActiveBrands	GET	Fetches list of available brands
getBrandDetails(name)	/webhook/GetBrandDetails	GET	Gets full brand configuration
analyzeKeyword(data)	/webhook/ AnalyzeKeyword_Backend	POST	Runs AI analysis, returns PAVE scores
saveIdea(data)	/webhook/SaveScoredIdea	POST	Saves scored keyword to SharePoint
getBacklog(params)	/webhook/FetchAllItems or FetchBriefedItem	GET	Gets content pipeline items (filtered by status)
getContentDetails(id)	/webhook/FetchItemDetails	GET	Gets full details for one item
recommendFormat(data)	/webhook/ BriefRecommendation	GET	AI recommends a content format
generateAuthorityBrief(data)	/webhook/ GenerateAuthorityBrief	POST	AI generates a full content brief
updateBriefStatus(data)	/webhook/ UpdateBriefAndStatus	PUT	Saves approved brief to SharePoint
produceContent(id)	/webhook/ProduceContent	POST	Generates full HTML article
publishContent(id)	/webhook/ UpdateContentStatus	PUT	Changes status to 'Published'

The Mock Data System

api.js has built-in sample data (MOCK_BRANDS, MOCK_BACKLOG, etc.) so the website can be tested without needing the n8n backend running. When VITE_USE_MOCK_DATA is 'true', every API call returns this fake data with a realistic delay to simulate real behaviour.

Tip: *This is really handy for frontend dev. A developer can work on the UI without worrying about whether the backend's available.*

10. Adding a New Backend Connection

If a new workflow gets added in n8n and you need the website to call it, here's what to do:

Step 1: Add the Method to api.js

Add to: src/services/api.js

```
// In src/services/api.js, add a new method to the ApiService class:

async myNewFunction(data) {
    if (USE_MOCK) {
        await delay(1000)  // Simulate loading time
        return { success: true, message: 'Mock response' }
    }
    // Real n8n call:
    return this.request('/webhook/MyNewWorkflow', {
        method: 'POST',
        body: JSON.stringify({
            field1: data.field1,
            field2: data.field2
        })
    })
}
```

Step 2: Call It from a Page

Use in: src/pages/[YourPage].jsx

```
// In any page file (e.g., src/pages/MyPage.jsx):

import { api } from '../services/api'

const handleClick = async () => {
    const result = await api.myNewFunction({ field1: 'value1' })
    console.log(result)
}
```

11. The Navigation System

Navigation has two parts that need to stay in sync:

Part	File	What It Controls
Routes	src/App.jsx	Which URL shows which page (the routing rules)
Sidebar Links	src/components/layout/Sidebar.jsx	The clickable links in the left sidebar

Current navigation links:

Name	URL Path
Dashboard	/dashboard
Analyze Keyword	/analyze
Content Backlog	/backlog
Create Brief	/brief
Produce Content	/produce
Published	/published

12. Adding a New Navigation Link

Same process as Section 7 (Adding a New Page), Step 3. Just add an entry to the navigation array in Sidebar.jsx.

One-line change:

```
// Add to the navigation array in src/components/layout/Sidebar.jsx:
{ name: 'Your New Page', href: '/your-url', icon: 'star' }
```

13. Reusable Components

These components are pre-built and can be dropped into any page. Just import and use.

LoadingSpinner

Shows a spinning animation with optional text. Use while data's loading.

How to use:

```
import LoadingSpinner from '../components/common/LoadingSpinner'

// Usage:
<LoadingSpinner text="Loading data..." />
<LoadingSpinner size="lg" text="Processing..." />
```

StatusBadge

Shows a coloured pill with the item's status.

How to use:

```
import StatusBadge from '../components/common/StatusBadge'

// Usage:
<StatusBadge status="Approved for Briefing" />
<StatusBadge status="Published" />
```

PAVEScoreCard

The full PAVE score display with editable inputs and save button.

How to use:

```
import PAVEScoreCard from '../components/common/PAVEScoreCard'

// Usage:
<PAVEScoreCard
    scores={results.pave_scores}
    searchSummary={results.search_summary}
    onSave={handleSave}
/>
```

CSS Classes You Can Use Anywhere

Class Name	What It Does	Usage Example
.card	A styled container box with border and padding	Content
.btn btn-primary	Purple filled button	<button className="btn btn-primary">Click</button>
.btn btn-secondary	Outline/ghost button	<button className="btn btn-secondary">Click</button>
.btn-lg	Makes a button larger	Add to existing button classes
.btn-full	Makes a button full-width	Add to existing button classes
.animate-fade-in	Fade-in animation on load	...
.page-header	Standard page title section	<h1>...</h1>
.page-title	Large page heading style	<h1 className="page-title">Title</h1>
.page-subtitle	Smaller subtitle under title	Description
.form-group	Groups a label + input together	...
.input	Styled text input field	<input className="input" />
.select	Styled dropdown select	<select className="select">...</select>
.label	Styled form label	<label className="label">Name</label>
.badge badge-success	Green status badge	Done
.badge badge-warning	Amber status badge	Pending
.table-container	Scrollable table wrapper	<table>...</table>
.empty-state	Centered empty message	No items

.stats-grid	Grid layout for stat cards	...
.stat-card	Individual statistic card	...

14. Environment Settings

Setting	Purpose	Current Value	Where to Change
VITE_N8N_API_URL	Where API calls go	/api/n8n	.env file + Vercel
VITE_N8N_API_KEY	n8n authentication key	(stored securely)	.env file + Vercel
VITE_USE_MOCK_DATA	Use fake data for testing?	false	.env file + Vercel

How it works: *Settings starting with VITE_ are special in Vite projects. They get baked into the website at build time. After changing a setting on Vercel, you'll need to redeploy for the change to take effect.*

15. Running the Website Locally

This section walks through everything a developer needs to get the website running on their own machine. We've included step-by-step instructions for Windows, Mac, and Linux since the commands are a bit different on each.

15.1 What You'll Need to Install

Before you can run the website locally, you need three things installed on your computer: Node.js (which includes npm), Git, and a code editor. Here's how to get each one.

Installing Node.js

Node.js is the runtime that lets you run JavaScript outside a browser. npm (Node Package Manager) comes bundled with it and is used to install all the project's dependencies. You want the LTS (Long Term Support) version, not the "Current" one.

On Windows:

Windows:

```
1. Go to https://nodejs.org
2. Click the big green LTS button to download the installer (.msi file)
3. Run the installer, click Next through all the steps
   - Leave all default options checked
   - Make sure "Add to PATH" is checked (it should be by default)
4. When it's done, open a NEW Command Prompt or PowerShell window
5. Verify it worked by typing:
```

Run these commands to verify (you should see version numbers):

```
node --version
npm --version
```

On Mac:

Mac:

```
# Option A: Download from the website (easiest)
1. Go to https://nodejs.org
2. Click the LTS button to download the .pkg installer
```

```
3. Run it, follow the prompts

# Option B: Using Homebrew (if you have it installed)
brew install node

# Verify it worked:
node --version
npm --version
```

On Linux (Ubuntu/Debian):

Linux (Ubuntu/Debian):

```
# Install Node.js using the NodeSource repository:
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs

# Verify it worked:
node --version
npm --version
```

Troubleshooting: *If you see version numbers like v20.x.x and 10.x.x, you're good. If you get "command not found", close and reopen your terminal, or restart your computer. The PATH sometimes doesn't update until you do.*

Installing Git

Git is what we use to download the project code and push changes back to GitHub. It's the version control system.

On Windows:

Windows:

```
1. Go to https://git-scm.com/download/win
2. The download should start automatically (.exe file)
3. Run the installer
   - You can accept all default settings
   - When it asks about the default editor, pick whatever you prefer
     (VS Code if you have it, otherwise Notepad is fine)
   - For "Adjusting your PATH", pick the recommended option
     ("Git from the command line and also from 3rd-party software")
4. Open a NEW Command Prompt or PowerShell window
5. Verify:
```

Should show something like: git version 2.x.x

```
git --version
```

On Mac:

Mac:

```
# Git might already be installed. Check first:
git --version

# If it's not installed, Mac will prompt you to install
# Xcode Command Line Tools. Click Install when prompted.
```

```
# Or install via Homebrew:
brew install git
```

On Linux (Ubuntu/Debian):

Linux:

```
sudo apt-get update
sudo apt-get install git

# Verify:
git --version
```

Installing a Code Editor

You need a text editor to view and edit the code. VS Code is the most popular choice and works on all three operating systems. It's free.

Editor	Download Link	Notes
VS Code (Recommended)	https://code.visualstudio.com	Free, works on Windows/Mac/Linux, tons of extensions
Cursor	https://cursor.sh	VS Code fork with built-in AI, great for AI-assisted coding
Sublime Text	https://www.sublimetext.com	Lightweight and fast, free to evaluate

Just download the installer for your OS from the website and run it. Nothing special to configure.

15.2 Opening a Terminal

You'll be typing commands into a terminal (also called command line or console). Here's how to open one on each OS:

Operating System	How to Open a Terminal
Windows	Press Win + R, type 'cmd' and hit Enter. Or search for 'PowerShell' in the Start menu. Or in VS Code: press Ctrl + ` (backtick)
Mac	Open Finder > Applications > Utilities > Terminal. Or press Cmd + Space, type 'Terminal' and hit Enter. Or in VS Code: press Ctrl + ` (backtick)
Linux	Press Ctrl + Alt + T. Or in VS Code: press Ctrl + ` (backtick)

Tip: *If you're using VS Code or Cursor, the built-in terminal (Ctrl + ` backtick) is the easiest option. It opens right inside the editor so you don't have to switch windows.*

15.3 Downloading and Running the Project

Now that you've got Node.js, Git, and a code editor installed, here's how to get the project running. These commands are the same on Windows, Mac, and Linux.

Step 1: Clone (download) the code from GitHub

Run this in your terminal:

```
git clone https://github.com/ifrahimafzal808-sudo/AI_Content.git
```

This creates a folder called AI_Content with all the project files inside it.

Step 2: Navigate into the frontend folder

On Windows, Mac, and Linux:

```
cd AI_Content/frontend
```

Windows note: *On Windows, you can also use backslashes: cd AI_Content\frontend. Both work.*

Step 3: Install all the project dependencies

This downloads everything the project needs:

```
npm install
```

This reads the package.json file and downloads all the required libraries into a folder called node_modules. It might take a minute or two the first time. You'll see a progress bar. Don't worry about any "warn" messages, those are usually fine.

Step 4: Set up your environment file

The project needs a .env file with some settings. There's a template file called .env.example that you can copy:

Copy the template:

```
# On Windows (Command Prompt):
copy .env.example .env

# On Windows (PowerShell):
Copy-Item .env.example .env

# On Mac/Linux:
cp .env.example .env
```

Then open the .env file in your code editor and fill in the values. The most important setting for local dev is:

Key settings in .env:

```
# Set this to 'true' to use fake test data (no backend needed)
VITE_USE_MOCK_DATA=true

# Or set to 'false' to use the real n8n backend
VITE_USE_MOCK_DATA=false
VITE_N8N_API_URL=/api/n8n
VITE_N8N_API_KEY=your_api_key_here
```

Step 5: Start the development server

Run this command:

```
npm run dev
```

You'll see output like this in the terminal:

```
  VITE v7.x.x  ready in xxx ms

  >  Local:   http://localhost:5173/
  >  Network: http://192.168.x.x:5173/
```

Step 6: Open the website in your browser

Open your web browser (Chrome, Firefox, Edge, Safari, whatever you use) and go to:

http://localhost:5173

You should see the AI Content Copilot landing page. That's it, you're running the website locally!

15.4 Day-to-Day Development Tips

Things that'll make your life easier while developing:

What	How
See changes instantly	The dev server watches your files. Save a file and the browser refreshes automatically. No need to restart anything
Test without the backend	Set VITE_USE_MOCK_DATA=true in your .env file. The website will use built-in fake data so you can work on the UI without needing n8n running
Stop the dev server	Go to your terminal and press Ctrl+C (works on all operating systems)
Restart the dev server	Press Ctrl+C to stop it, then run 'npm run dev' again
Install a new library	Run 'npm install library-name' (e.g., npm install axios). This updates package.json automatically
Build for production	Run 'npm run build' to create the optimized files that go to Vercel. The output goes into a 'dist' folder
Check for errors	Open your browser's developer tools (F12 on most browsers > Console tab) to see JavaScript errors

15.5 Common Problems When Setting Up Locally

Problem	Cause	Fix
'node' is not recognized / command not found	Node.js isn't installed or not in your PATH	Reinstall Node.js, make sure 'Add to PATH' is checked. Close and reopen your terminal after installing
'git' is not recognized / command not found	Git isn't installed or not in your PATH	Reinstall Git, accept the default PATH option. Close and reopen your terminal
npm install shows errors	Usually network issues or permission problems	Try running as administrator (Windows) or with sudo (Mac/Linux). Or try 'npm cache clean --force' first
Port 5173 is already in use	Another dev server or app is using that port	Either close the other app, or run 'npm run dev -- --port 3000' to use a different port
Blank page after npm run dev	Usually a JavaScript error or wrong .env config	Check the terminal for errors. Check the browser console (F12). Make sure .env file exists
API calls fail locally	Mock mode is off but backend isn't accessible	Set VITE_USE_MOCK_DATA=true in .env for local testing, or make sure n8n Cloud is running

16. How Deployments Work

- 1. Developer makes changes locally
- 2. Developer pushes to GitHub ('git push')
- 3. Vercel picks up the push automatically
- 4. Vercel runs 'npm run build' to compile

- 5. If successful, the new version goes live
- 6. If it fails, the old version stays up (no downtime)

Safety net: *Vercel never takes down the old version until the new one's confirmed working. If a deployment fails, your website just keeps running the previous version. There's zero risk of downtime from a bad deploy.*

17. Maintaining the App with AI

Because the project's using standard React with clean, well-organized code, AI coding assistants do really well with it.

AI Coding Tools Worth Looking At

Tool	Best For	Cost
Claude (Anthropic)	Complex code changes, understanding architecture	Free tier available, Pro ~$20/month
GPT-5 (OpenAI / ChatGPT)	General code changes, explanations	Free tier available, Plus ~$20/month
Cursor (IDE)	Full dev environment with built-in AI	Free tier, Pro ~$20/month
GitHub Copilot	Inline code suggestions while typing	~$10-19/month
Lovable	No-code frontend generation through prompting	Subscription-based, good for simpler UIs

How to Prompt an AI Assistant

When asking an AI to make changes, include these details:

Tell it the stack: *"This is a React + Vite project using vanilla CSS"*

Point to the file: *"Edit src/pages/AnalyzeKeyword.jsx"*

Say what you want: *"Add a text field called 'Notes' below the keyword input"*

Mention styling: *"Use the existing CSS classes: className='form-group' for the wrapper and className='input' for the field"*

Give context: *"The API service is in src/services/api.js and all styles are in src/index.css"*

Example Prompt:

```
EXAMPLE PROMPT:

"I've got a React + Vite frontend for an AI content platform.
Here's the structure:
  - Pages are in src/pages/
  - API calls are in src/services/api.js
  - Styles are in src/index.css
  - Routes are defined in src/App.jsx
  - Sidebar links are in src/components/layout/Sidebar.jsx

I want to add a new page called 'Analytics' at /analytics that
```

```
shows a chart of how many articles were published per week.
Please create the page file, add the route, and add the sidebar link."
```

18. Quick Reference Cheat Sheet

Keep this page handy:

I Want To...	Edit This File	Section
Change the website colours	src/index.css (CSS variables at top)	Section 5
Change a page's title or text	src/pages/[PageName].jsx	Section 8
Add a brand-new page	New file in src/pages/ + App.jsx + Sidebar.jsx	Section 7
Add a sidebar link	src/components/layout/Sidebar.jsx	Section 12
Connect a new n8n workflow	src/services/api.js	Section 10
Change the navbar logo/title	src/components/layout/Navbar.jsx	Section 8
Toggle mock data on/off	.env file > VITE_USE_MOCK_DATA	Section 14
Change the n8n backend URL	.env + vercel.json + vite.config.js	Section 14
Deploy a new version	Push code to GitHub (automatic)	Section 16
Force a redeployment	Vercel dashboard > Deployments > Redeploy	Section 16